

Rapport TER

Approche moderne du rendu différentiel

2026

Corentin VAILLANT, Julien LEFEBVRE

Université de Toulouse

2026

L3 MIDL

Encadrant : Mathias PAULIN

Table des matières

1	Introduction	3
2	Le Rendu	4
2.1	L'équation du rendu	4
2.2	Par quadrature (Ray Tracing)	4
2.3	Le Path Tracing	6
2.4	Autres Méthodes	8
3	La Différentiation	10
3.1	La Méthode Naïve	10
3.2	La Méthode Etudiée (Méthode de l'UC)	10
3.3	Autres Méthodes (Methode de l'EPFL)	12
4	Implantation des algorithmes	13
4.1	Premier ray tracer	13
4.2	Autres implantation CPU	13
4.3	Passage sur GPU	14
5	Conclusion	16
6	Annexe	17
6.1	Bibliographie	17

1 | Introduction

Le rendu différentiel est une branche de l'informatique graphique où l'on cherche à trouver la différentielle de l'image d'un rendu d'une scène, ce qui est très utile dans de nombreux domaines. Il possède de nombreuses applications, notamment dans le rendu plus classique, car il permet de faire varier des paramètres de la scène sans obligation de rendre la scène une nouvelle fois de zéro, ce qui peut être très coûteux. Il est aussi utile dans le rendu physiquement réaliste parce qu'il a permis de résoudre des problèmes d'analyse-par-synthèse dans des domaines comme le rendu de vêtements ou encore la création de matériaux translucides. Son utilité ne se limite pas qu'au rendu, il possède aussi des applications dans le domaine du machine learning car il permet d'entraîner des réseaux de neurones de manière plus efficace. Le rendu différentiel a une communauté de recherche très active. En effet ce domaine est très challengeant parce qu'à ce jour aucun algorithme efficace n'introduisant pas de biais n'a été découvert. Cela est notamment dû au fait de l'absence d'estimateurs de Monte Carlo efficaces.

Dans ce TER, nous nous sommes intéressés aux travaux de Cheng Zhang, Bailey Miller, Kai Yan, Ioannis Gkioulekas et Shuang Zhao (**Path-Space Differentiable Rendering**) qui proposent une solution sans biais à ce problème basée sur la séparation en deux sous-problèmes.

Pour effectuer ce TER, nous nous sommes aussi intéressés au ray tracing ainsi qu'au path tracing, notamment grâce à la série de livres **Raytracing in one weekend**, dans le but d'acquérir les bases nécessaires à la compréhension du papier.

Dans le cadre de ce travail, nous avons aussi tenté d'implanter un Path Tracer sur GPU avec Vulkan (**Github vers les projet**).

Nous allons introduire dans un premier temps le ray tracing ainsi que ses limites. Puis nous regarderons ce qu'est le Path Tracing et comment il corrige les limites du Ray Tracing. Dans un troisième temps, nous parlerons du rendu différentiel et de la méthode proposée par Shuang Zhao et son équipe. Enfin nous finirons par regarder une autre approche à ce problème avec les travaux de Tizian Zeltner, Sébastien Speierer, Iliyan Georgiev et Wenzel Jakob (**Monte Carlo Estimators for Differential Light Transport**).

2 | Le Rendu

2.1 | L'équation du rendu

Pour effectuer un rendu 3D, nous allons essayer de calculer la luminance des différentes surfaces de notre scène. La luminance représente la luminosité que l'œil humain perçoit, provenant d'une surface qui émet de la lumière soit en tant que source de lumière, soit par réflexion ou transmission. L'équation du rendu (**Page Wikipedia Equation du Rendu**) nous permet de calculer la luminance sur les différentes surfaces de notre scène 3D. On peut la définir comme ci-dessous :

Définition 2.1. (Equation du Rendu)

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} L_i(x, \omega_i) f_i(x) \overline{\cos(\theta_i)} d\omega_i \quad (1)$$

- $L_o(x, \omega_o)$ représente la luminance sortante au point x dans la direction ω_o .
- $L_e(x, \omega_o)$ représente la luminance émise par le point x dans la direction ω_o .
- Ω représente la sphère de rayon 1 autour du point x .
- $L_i(x, \omega_i)$ représente la luminance arrivant en x depuis la direction ω_i et elle est définie de la manière suivante : $L_i(x, \omega_i) = L_o(vis(x, \omega_i), -\omega_i)$ où $vis(x, \omega_i)$ donne le premier point intersecté en partant dans la direction ω_i à partir du point x .
- $f_i(x)$ représente la BRDF qui nous donne la distribution de la luminance arrivant en x depuis la direction ω_i et réfléchi dans la direction ω_o , il s'agit en faite d'une fonction décrivant les propriétés photométriques d'une matière.
- θ représente l'angle formé ω_i et la $\vec{n}$ normale de la surface.
- $\overline{\cos(\theta)} = \cos(\theta)$ si $\cos(\theta) > 0$ sinon 0.

L'un des problèmes de cette équation est que l'on ne peut pas résoudre l'intégrale de façon analytique, notamment en raison de sa nature récursive. C'est pour cela que des méthodes pour estimer cette intégrale ont été mises en place.

2.2 | Par quadrature (Ray Tracing)

Définition 2.2. (Rayon) On appelle rayon un couple $\vec{r} = (o, \vec{d}) \in \mathbb{R}^3 \times \mathbb{U}(\mathbb{R}^3)$ où o est un point désignant l'origine du rayon et $\vec{d}$ est un vecteur unitaire désignant la direction du rayon. À noter que la direction n'est pas nécessairement unitaire, mais nous la prendrons ainsi sans perte de généralité pour des raisons de simplicité dans le reste de ce rapport.

Le ray tracing (lancée de rayon dans la langue de Molière) est un algorithme permettant d'estimer l'équation du rendu (équ. 1). Il se base sur une technique de lancer de rayon à partir de la caméra vers la scène, c'est-à-dire dans le sens inverse de la lumière. Cela donne le même résultat que dans le sens de la lumière d'après le principe du retour inverse de la lumière de Fermat. Il se base aussi sur les lois de Snell-Decartes de réflexion et réfraction de la lumière. Il s'agit en fait d'une quadrature de la scène, à partir de rayon envoyer récursivement. Une méthode de ray tracing naïve de la marche aléatoire :

Algorithm 1 . – Marche Aléatoire

```

1: procedure  $L_i(\text{Rayon } \vec{r}, \text{depth} \in \mathbb{N})$ 
2:    $\text{intersection} \leftarrow \text{Intersection}(\vec{r})$ 
3:    $\triangleright$  Cas où le rayon n'intersecte aucune surface
4:   if  $\neg \text{intersection}$  then
5:     return  $\text{LumièresDirectionelles}.L_e(\vec{r})$ 
6:   end
7:    $\triangleright$  Recupération de la surface intersectée et de son émission
8:    $\text{surface} \leftarrow \text{intersection.surface}$ 
9:    $\omega_o \leftarrow -\vec{r} \cdot \vec{d}$ 
10:   $L_e \leftarrow \text{surface}.Le(\omega_o)$ 
11:   $\triangleright$  Cas où on a atteint le nombre maximum d'itération
12:  if  $\text{depth} = \text{maxDepth}$  then
13:    return  $L_e$ 
14:  end
15:   $\triangleright$  Calcul de la partie non récursive de l'intégrale de l'équation du rendu
16:   $\text{bsdf} \leftarrow \text{surface}.BSDF$ 
17:   $\omega_i \leftarrow \mathcal{U}(\Omega)$ 
18:   $f_{\cos} \leftarrow \text{bsdf}(\omega_o, \omega_i) \times |\omega_i \cdot \vec{n}|$ 
19:   $\triangleright$  Si la partie non récursive est nulle il est inutile de continuer d'itérer
20:  if  $f_{\cos} = 0$  then
21:    return  $L_e$ 
22:  end
23:   $\triangleright$  Appel récursif, ( $\frac{1}{4\pi}$  : Probabilité de tiré une direction.)
24:   $\vec{r} \leftarrow (\text{intersection.point}, \omega_i)$ 
25:  return  $L_e + f_{\cos} \times L_i(\vec{r}, \text{depth} + 1) \div (\frac{1}{4\pi})$ 
26: end

```

Généralement, les implémentation du lancer de rayon son avec de meilleur moyen pour estimer le prochain rayon que la marche aléatoire, afin d'obtenir une meilleur convergence.

2.3 | Le Path Tracing

L'équation du rendu (éq. 1) peut être réécrite de façon à enlever son aspect récursif. Pour cela, au lieu d'intégrer sur la sphère unitée autour de chacun des points, nous allons effectuer un changement de variable et intégrer sur des chemins. Cette réécriture a été proposée par Veach en 1997 notamment dans son papier **Metropolis Light Transport**.

Définition 2.3. (Chemin de lumière et espace des chemins) Soit $\mathcal{M}$ l'ensemble des surfaces des objets de notre scène. On appelle chemin de lumière (ou light path) un vecteur $\bar{x} = (x_0, x_1, \dots, x_N)$ de $\mathcal{M}^{N+1}$. Les chemins commencent sur une lumière en x_0 pour aller jusqu'à la caméra en x_N . On appelle espace des chemins (ou path space) l'ensemble $\Omega := \cup_{N=1}^{\infty} \mathcal{M}^{N+1}$.

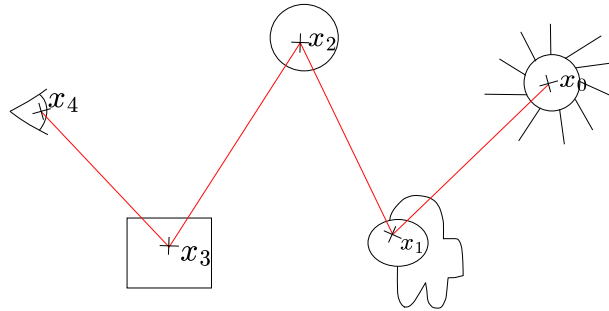


Fig. 1 . – Exemple de chemin de $\mathcal{M}^{N+1}$

En réécrivant l'équation du rendu (éq. 1) sur l'espace des chemins, on obtient:

Définition 2.4. (Équation du rendu sur l'espace des chemins)

$$I = \int_{\Omega} f(\bar{x}) d\mu(\bar{x}) \quad (2)$$

Où μ est la mesure du produit des aires défini par $d\mu(\bar{x}) := \prod_{n=0}^N dA(x_n)$ avec A la mesure de l'aire d'une surface et $f(\bar{x})$ est défini de la façon suivante :

$$f(\bar{x}) = \left(\prod_{n=0}^{N-1} g(x_{n+1} : x_n, w_n) \right) \cdot W_e(x_N \rightarrow x_{N-1}) \quad (3)$$

où W_e est l'importance du capteur dans notre cas, W_e sera une constante égale à 1 car on utilise une caméra trou d'épingle et

$$g(x_{n+1} : x_n, w_n) := f_s(x_n \rightarrow x_{n+1}) \cdot G(x_n, x_{n+1}) \quad (4)$$

où f_s est la BSDF au point x_n dans la direction arrivant de x_{n-1} et allant vers x_{n+1} si $n > 0$ sinon $f_s := L_e(x_0 \rightarrow x_1)$ où L_e est l'émission de la surface x_0 dans la direction de x_1 et

$$G(x_n, x_{n+1}) := \mathbb{V}(x_n, x_{n+1})G_0(x_n, x_{n+1}) \quad (5)$$

où $\mathbb{V}(x_n, x_{n+1})$ vaut 1 si x_n et x_{n+1} sont visibles, c'est-à-dire s'il n'y a pas de surfaces opaques entre eux, et 0 sinon et

$$G_0(x_n, x_{n+1}) := \frac{|\vec{n}_{x_n} \cdot \omega_n| |\vec{n}_{x_{n+1}} \cdot (-\omega_n)|}{\|x_{n+1} - x_n\|^2} \quad (6)$$

où ω_n représente le vecteur unitaire partant x_n et pointant vers x_{n+1} .

Cette réécriture de l'équation du rendu sur l'espace des chemins est ce qui nous permettra dans la suite, lorsque nous aborderons les travaux de Shung Zhao et son équipe (**Path-Space Differentiable Rendering**) de calculer la différentielle de notre scène. En effet, on verra qu'il est possible de « rentrer » la différentielle à l'intérieur de l'intégrale et que cela nous créera deux intégrales, une intérieure (ou interior) et une de bord (ou boundary).

Cette réécriture est aussi à la base de l'algorithme du path tracing qui est fondamental car nous verrons qu'il est possible de le différencier. Il se base sur une méthode de Monte Carlo pour estimer l'intégrale sur les chemins (éq. 2).

Algorithm 2 . – Path Tracer

```

1: procedure  $L_i(\text{Rayon } \vec{r})$ 
2:    $\omega_o \leftarrow \vec{r} \cdot \vec{d}$ 
3:    $x_1 \leftarrow \vec{r} \cdot o$ 
4:   intersection  $\leftarrow \text{Intersection}(\vec{r})$ 
5:   if  $\neg \text{intersection}$  then
6:     return LumièresDirectionelles. $L_e(\vec{r})$ 
7:   end
8:    $x_2 \leftarrow \text{intersection.point}$ 
9:    $L \leftarrow \text{intersection}.L_e(\omega_o)$ 
10:   $\beta \leftarrow 1$ 
11:   $\triangleright \varepsilon > 0$  fixé.
12:  while  $\beta > \varepsilon$  do
13:     $\triangleright$  Éclairage direct (échantillonnage de la lumière)
14:     $p_0 \leftarrow p \sim \mathbb{P}_{\text{lumières}}$ 
15:     $\omega_i \leftarrow \frac{p_0 - x_2}{\|p_0 - x_2\|}$   $\triangleright$  direction de  $x_2$  vers  $p_0$ 
16:     $\alpha_{\text{directe}} \leftarrow p_0 \cdot L_e(p_0 \rightarrow x_1) f_s(p_0 \rightarrow x_1 \rightarrow x_2) G(p_0, x_1)$ 
17:     $L \leftarrow L + \beta \alpha_{\text{directe}} / \mathbb{P}_{\text{lumières}}(p_0)$ 
18:  
```

```

19:    $\omega_i \leftarrow \omega \sim \mathbb{P}_{\text{bsdf}}$ 
20:    $\triangleright (x_1, \omega_i)$  : le rayon partant de  $x_1$  avec la direction  $\omega_i$ 
21:   intersection  $\leftarrow$  Intersection( $(x_1, \omega_i)$ )
22:   if  $\neg$ intersection then
23:     BREAK
24:   end
25:    $\alpha_{\text{indirecte}} \leftarrow f_s(x_0 \rightarrow x_1 \rightarrow x_2)G(x_0, x_1)$ 
26:    $\triangleright$  On convertie la probabilité en mesure d'aire :
27:    $q \leftarrow \mathbb{P}_{\text{bsdf}}(\omega_i) \frac{|n(x_0) \cdot -\omega_i|}{\|x_0 - x_1\|^2}$ 
28:    $\beta \leftarrow \beta \alpha_{\text{indirecte}} / q$ 
29:
30:    $\triangleright$  On continue le chemin.
31:    $(x_2, x_1) \leftarrow (x_1, x_0)$ 
32:    $\vec{x} \leftarrow (x_1 \rightarrow x_2)$ 
33: end
34: return  $L$ 
35: end

```

2.4 | Autres Méthodes

De nombreuses autres méthodes existent pour résoudre l'équation du rendu et ont chacune leurs avantages mais aussi leurs inconvénients. Parmi celles-ci, on retrouve :

1. La rasterisation qui, comme le ray tracing, est une quadrature de l'équation du rendu. Elle est non biaisée et différentiable mais elle reste très limitée, notamment au niveau de la différenciation et à une convergence assez lente.
2. Le photon mapping qui se base aussi sur une technique de lancer de rayon. Contrairement au ray tracing et au path tracing, les rayons sont envoyés depuis les lumières et on enregistre leurs chemins dans une photon map. Puis pour effectuer le rendu, on va envoyer un rayon depuis la caméra et à son point d'intersection avec une surface, regarder la concentration de points à proximité sur la photon map avec un algorithme des plus proches voisins. Cet algorithme est biaisé, donc il n'a pas vraiment d'intérêt à être différencié, mais il joue un rôle dans la méthode proposée par Shuang Zhao et son équipe (**Path-Space Differentiable Rendering**), nous en reparlerons dans la section dédiée.
3. Le bidirectional path tracing est une version améliorée du path tracing. Au lieu de créer des chemins uniquement depuis la caméra, il va aussi créer des chemins partant des lumières et les connecter entre eux avec un rayon de visibilité. De plus cet algorithme est non biaisé mais, malheureusement, à ce jour aucune technique de différenciation n'a été découverte.

4. Bien d'autres méthodes existent comme le **Metropolis Ligth Transport** qui se base sur l'algorithme de Metropolis-Hastings, le **Vertex Connection and Merging** qui fusionne le bidirectional path tracing et le photon mapping, le **Cone Tracing** qui donne une épaisseur aux rayons, ce que ne fait pas le raytracing ou encore le **Gaussian Splating** qui effectue le rendu à l'aide de données extraites d'image et beaucoup d'autres...

3 | La Différentiation

3.1 | La Méthode Naïve

Pour obtenir une approximation, nous pourrions utiliser la méthode la différence fini.

Pour toute fonction continue f , le théorème de Taylor nous donne pour V un voisinage de x_0 :

$$\begin{aligned} h : x_0 + h &\in V \\ f(x_0 + h) &= \sum_{n \in \mathbb{N}} \frac{f^{(n)}(x_0)}{n!} h^n \\ &= f(x_0) + f'(x_0)h + R(x_0 + h) : \text{Avec } R(x_0 + h) \text{ le reste.} \\ \Leftrightarrow \frac{f(x_0 + h) - f(x_0)}{h} &= f'(x_0) + \frac{R(x_0 + h)}{h} \\ \Leftrightarrow f'(x_0) &= \frac{f(x_0 + h) - f(x_0)}{h} - \frac{R(x_0 + h)}{h} \end{aligned}$$

En assumant $R(x_0 + h)$ suffisamment petit, nous avons donc :

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0)}{h}$$

Nous pouvons donc appliquer cette méthode, en utilisant h assez petit.

Ici $\pi \in \mathbb{R}$ représente un paramètre de la scène. On notera toutes les fonctions dépendant de la scène à l'aide de la syntaxe suivantes $\varphi(\pi : x_1, \dots, x_k)$, où φ est une fonction quelconque dépendant des paramètres de la scène.

Nous avons donc :

$$\frac{\partial}{\partial \pi} \int_{\Omega} f(\pi : \bar{x}) d\mu(\bar{x}) \simeq \frac{\int_{\Omega} f(\pi : \bar{x}) d\mu(\bar{x}) - \int_{\Omega} f(\pi - h : \bar{x}) d\mu(\bar{x})}{h}$$

La méthode est donc facile à comprendre et à implanter, mais elle souffre de deux gros inconvénients. Premièrement, la stabilité numérique des nombres à virgule flottantes, en effet, s'il on prend h trop petit, les imprécisions dans le calcul deviendront de plus en plus importantes.

Et de plus, cette méthode nécessite de faire deux rendus, ce qui provoque donc le doublage du temps de calcul.

3.2 | La Méthode Étudiée (Méthode de l'UC)

Avant de commencer à aborder la méthode de Shuang Zhao et son équipe (**Path-Space Differentiable Rendering**) nous allons d'abord introduire quelques définitions importantes.

Définition 3.1. (Configuration de référence, mouvement et déformation) Soit $\mathcal{B}$ un manifold 2D abstrait que l'on nommera configuration de référence. On appelle une déformation de $\mathcal{B}$ une fonction injective différentiable de $\mathcal{B}$ sur une

surface $\mathcal{M}$. On appelle X un mouvement de $\mathcal{B}$ une fonction $\mathcal{C}^3$ de $\mathcal{B} \times \mathbb{R}$ dans $\mathcal{M}$. De plus nous pouvons remarquer que si on fixe un paramètre $\pi \in \mathbb{R}$, alors $X(\cdot, \pi)$ est une déformation de $\mathcal{B}$.

Définition 3.2. (Surface évoluant, carte de référence et trajectoire) Pour un mouvement X et un paramètre π , on appelle surface évoluant l'image $\mathcal{M}(\pi) := \{X(p, \pi) : p \in \mathcal{B}\} \subset \mathbb{R}^3$ de la fonction $X(p, \pi)$. De plus, comme cette fonction est injective, on peut définir une fonction inverse sur son image $P(\cdot, \pi) : \mathcal{M}(\pi) \rightarrow \mathcal{B}$ que l'on appellera carte de référence. Enfin on appellera trajectoire l'ensemble $\mathcal{T}$ des couples $(x, \pi) \in \mathcal{M}(\pi) \times \mathbb{R}$ des surfaces évoluantes et de leur paramètre π associé.

Définition 3.3. (Espace des matériaux et espace des position) On appellera $p \in \mathcal{B}$ un point de matériau et $x \in \mathcal{M}(\pi)$ un point de position. Nous pouvons aussi remarquer que la déformation $X(\cdot, \pi)$ établit une bijection entre les points de matériau et les points de position. On appellera champ de matériau une fonction de $\mathcal{B} \times \mathbb{R}$ et champ de position une fonction sur la trajectoire $\mathcal{T}$.

Pour les définitions nous allons poser $\varphi(x, \pi)$ un champ de position scalaire sur $\mathcal{M}(\pi)$.

Définition 3.4. (Dérivée de scène) φ admet une dérivée de scène de la forme :

$$\dot{\varphi}(x, \pi) = \frac{\partial}{\partial \pi'} \varphi(\hat{x}(\xi, \pi'), \pi') \big|_{\pi'=\pi} \quad (7)$$

Elle admet aussi une forme normalisée de cette dérivée :

$$\varphi^\square = \dot{\varphi} - v_{\tan} \cdot \text{grad}_{\mathcal{M}}(\varphi) \quad (8)$$

Définition 3.5. (Courbes de discontinuité et bordure étendue) La fonction φ est $\mathcal{C}^0$ en fonction de x sauf sur un ensemble de courbes qui évolue de façon continue en fonction de π . On pose $\Delta\mathcal{M}[\varphi](\pi) \subset \mathcal{M}(\pi)$ l'ensemble des ces courbes de discontinuité. On appellera $\overline{\partial\mathcal{M}}[\varphi](\pi)$ la bordure étendue l'ensemble des bordures de $\mathcal{M}(\pi)$ et des courbes de discontinuité.

En utilisant la relation de transport venant de la mécanique des fluides par *Cermelli* (**Transport relations for surface integrals arising in the formulation of balance laws for evolving fluid interfaces**) on peut obtenir :

$$\frac{d}{d\pi} \int_{\mathcal{M}} \varphi dA = I_{\text{intérieur}} + I_{\text{bordure}} \quad (9)$$

Avec

$$I_{\text{intérieur}} = \int_{\mathcal{M}} (\varphi^{\square} - \varphi \kappa V) dA$$

$$I_{\text{bordure}} = \int_{\partial \mathcal{M}} \Delta \varphi V_{\partial \mathcal{M}} dA$$

Avec κ la courbature totale, V et $V_{\partial \mathcal{M}}$ les vistsesses normales scalaires de $\mathcal{M}(\pi)$ et de la bordure étendue et $\Delta \varphi(x, \pi)$ le prolongement défini comme si dessous :

$$\Delta \varphi(x, \pi) := \begin{cases} \varphi(x, \pi) & \text{si } x \in \partial \mathcal{M}(\pi) \\ \varphi^-(x, \pi) - \varphi^+(x, \pi) & \text{si } x \in \Delta \mathcal{M}(\pi) \end{cases}$$

Algorithm 3 . – Differentiation du Path Tracer : bordure

```

1: procedure PT_diff_intérieur( $\mathbb{P}$ , Bool direct)
2:   ▷ Échantillonnage d'un segment de bordure
3:    $(x^B, \omega^B) \leftarrow (x, \omega) \sim \mathbb{P}(x, \omega)$ 
4:    $\text{intersec}_{\text{src}} \leftarrow \text{Intersection}((x^B, -\omega^B))$ 
5:    $\text{intersec}_{\text{cam}} \leftarrow \text{Intersection}((x^B, \omega^B))$ 
6:   ▷ Si les deux rayons tapent
7:   if  $\text{intersec}_{\text{src}} \wedge \text{intersec}_{\text{cam}}$  then
8:     |  $\beta^B \leftarrow \tilde{f}^B$ 
9:   end
10: end

```

3.3 | Autres Méthodes (Methode de l'EPFL)

4 | Implantation des algorithmes

Afin de pouvoir comprendre les algorithmes étudiés, nous avons implémenté un ray tracer et un path tracer.

4.1 | Premier ray tracer

Nous avons donc commencé par suivre une série de tutoriels : **Raytracing in one weekend**, écrit principalement par *Peter Shirley*.

Le but de ces tutoriels était d'implanter, en C++, un premier ray tracer.

Cela nous a permis de comprendre le ray tracing de façon concrète, nous avons écrit plusieurs abstractions (formes, matériaux, textures, caméra, *ect.*), qui nous ont ensuite permis d'appliquer les mathématiques de l'algorithme.

Cette première implantation souffrait de la faible parallélisation du CPU. En effet, les temps de rendu pouvaient être très longs.

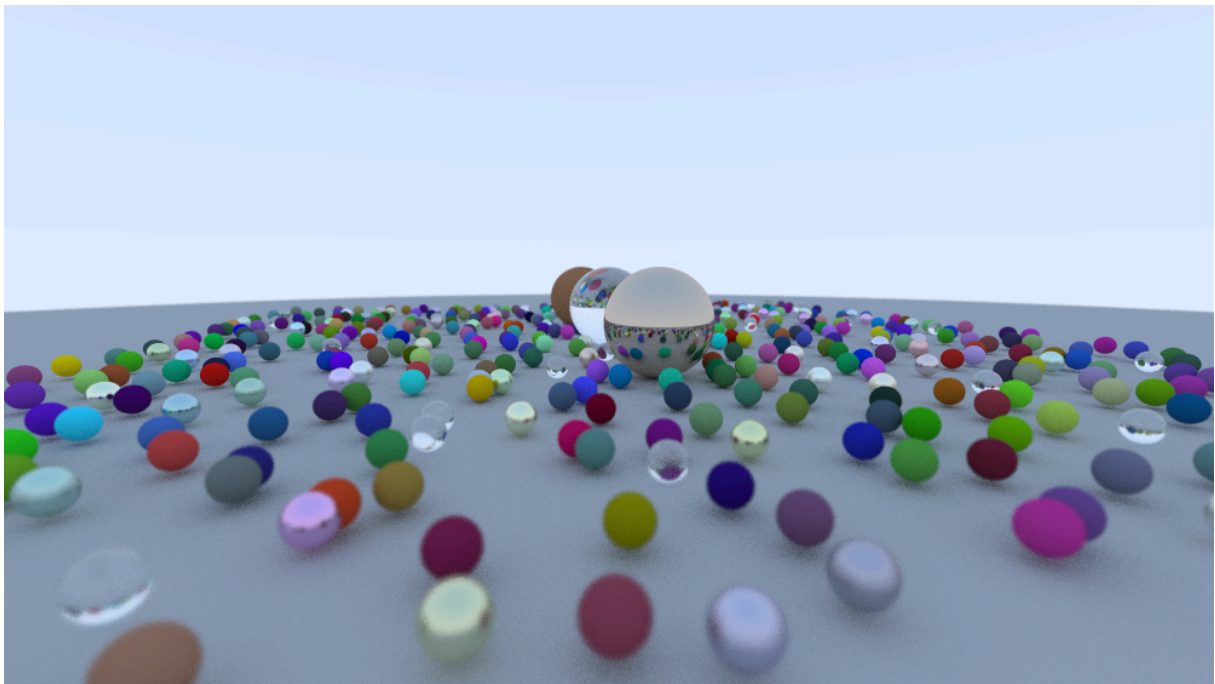


Fig. 2 . – Scène rendue à partir du ray tracer implanté, le temps de rendu est d'environ 3 minutes (CPU 3.5 GHz).

4.2 | Autres implantations CPU

La suite des tutoriels de *Peter Shirley* nous a amenés à construire des rendus plus élaborés. Nous avons donc ajouté la possibilité d'avoir des matériaux volumétriques et des textures.

TODO IMG VOLUMÉTRIQUE && textures

Afin d'accélérer le processus, nous avons implanté divers moyens d'accélérer le processus, comme des **BVH** (Bounding Volume Hierarchy, comprendre hiérarchie des volumes englobants), et des méthodes de Monte Carlo plus efficaces, par exemple, au lieu d'opérer la quadrature sur une marche aléatoire à direction uniforme, nous tirons une direction en fonction de la distribution de la **BRDF**.

Au fur et à mesure, le tutoriel nous amène à construire un path tracer, les nouvelles directions sont donc tirées en fonction du placement des lumières, et l'on passe d'un programme récursif à terminal-récursif.

De plus, nous avons ajouté quelques fonctionnalités supplémentaires, telles que le chargement de maillage et le multi-threading. **TODO add mesh**

4.3 | Passage sur GPU

Pour la suite nous avons décidé d'utiliser **Vulkan** pour une implantation sur GPU. **Vulkan** est une spécification proposée par *Khronos Group* (qui propose aussi **OpenGL**), qui a vocation à remplacer **OpenGL**. **Vulkan** nous permet d'avoir un grand contrôle sur la carte graphique, mais comme le dit l'adage : « Un grand pouvoir implique de grandes responsabilités », la spécification est donc dure à prendre en main, avec un débogage compliqué, et une quantité de code à écrire conséquente. Afin de nous aider, nous avons encore une fois suivi ce tutoriel : **VulkanGuide**, qui nous a permis d'implanter un simple moteur graphique de rasterization. Beaucoup d'abstractions utilisées dans le tutoriel ont été réutilisées par la suite.

Une fois **Vulkan** appréhendé, nous avons fait notre premier ray tracer sur GPU, bien que rudimentaire, ce dernier était beaucoup plus rapide que les implantations sur CPU.

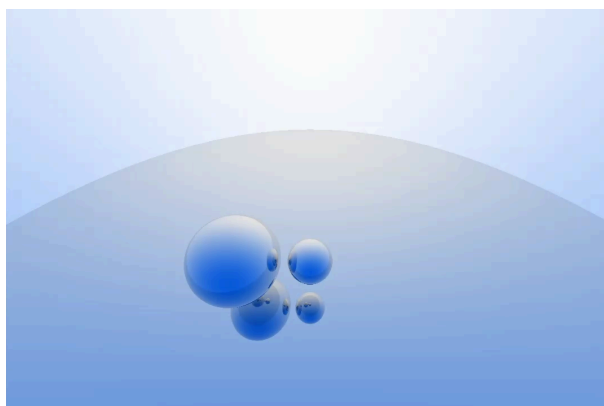


Fig. 3 . – Scène rendue à partir du ray tracer fait sur ``Vulkan``, rendu en temps réel (carte graphique intégrée sur un CPU).

De plus, nous avons utilisé le langage de shader **Slang**, un langage de shader qui nous permet d'exécuter du code directement sur la carte graphique. Nous avons choisi ce langage car l'auto-différentiation y est implanté.

Malheureusement, le temps nous a manqué, l'implantation du ray tracer sur Vulkan a pris plus de temps que prévu, et la complexité des mathématiques utilisées dans l'algorithme du path tracer nous a ralenti. Le path tracer que nous voulions implémenter n'a donc pas pu être fini à temps.

Nous avons donc décidé qu'après le rendu de ce rapport, nous finaliserons l'implantation, pour à terme, y inclure les travaux de *Shuang Zhao*.

Le code pourra être trouvé sur le repo Github **Vulkan Path Tracer**.

5 | Conclusion

6 | Annexe

6.1 | Bibliographie